ПРАВИТЕЛЬСТВО САНКТ-ПЕТЕРБУРГА
КОМИТЕТ ПО НАУКЕ И ВЫСШЕЙ ШКОЛЕ

Санкт-Петербургское государственное бюджетное профессиональное образовательное учреждение «ПЕТРОВСКИЙ КОЛЛЕДЖ»
(СПб ГБПОУ «Петровский колледж»)
Отделение информационных технологий

КУРСОВОЙ ПРОЕКТ

ПМ.02 Сопровождение и схемотехническое обслуживание интеллектуальных интегрированных систем
МДК.02.01 Сопровождение и схемотехническое обслуживание интеллектуальных интегрированных систем

РАЗРАБОТКА И СОПРОВОЖДЕНИЕ ВИРТУАЛЬНОЙ СИСТЕМЫ СБОРА И УЧЁТА ДАННЫХ НА ARDUINO «ВАРИАНТ №16»

Выполнил:

Моисеев Алексей Алексеевич

студент:	54-15 группы
Специальность:	09.02.08 Интеллектуальные интегрированные системы
Руководитель:	Борисов Е.Н., Дадаев М.В.
Оценка	

Санкт-Петербург, 2026

ПРАВИТЕЛЬСТВО САНКТ-ПЕТЕРБУРГА
КОМИТЕТ ПО НАУКЕ И ВЫСШЕЙ ШКОЛЕ
СПб ГБПОУ «Петровский колледж»
Отделение информационных технологий

Задание для выполнения курсового проекта

Специальность: 09.02.08 «Интеллектуальные интегрированные системы»
Группа: 54-15 Ф.И.О. студента: Моисеев Алексей Алексеевич
Тема: «Разработка и сопровождение виртуальной системы сбора и учёта данных на Arduino. Вариант №16».

Выполнить структурную и принципиальную схемы, модель в Wokwi и прошивку Arduino Uno; спроектировать SQLite, организовать загрузку данных средствами Node-RED, провести испытания и вести проект в Git.

Таблица — Исходные данные варианта №16

Параметр	**Значение**
Сценарий	Гараж
Датчик 1 (аналоговый)	MQ-2; условная шкала CO 0–300 ppm
Датчик 2	PIR; движение 0/1
Датчик 3	SW1; положение ворот 0/1
Исполнительное устройство	Реле вытяжки
Светодиод-индикатор	Красный, 10 мА
Период опроса	T = 5 с
Порог события	CO > 50 ppm

В дополнение к варианту реализован переключатель SW2 для выбора режима AUTO/TEST. Это не изменяет назначенный порог и период опроса.

Дата выдачи: «13» октября 2026 г. Срок защиты: «05» ноября 2026 г.
Руководитель: Борисов Е.Н., Дадаев М.В. Студент: Моисеев А.А.

Содержание

Введение 4
1. Постановка задачи и структурная схема 5
2. Принципиальная схема в EasyEDA и схема в Wokwi 6
3. Расчётная часть 7
3.1. Расчёт резистора светодиода 7
3.2. Расчёт шага АЦП 7
3.3. Расчёт объёма данных 7
4. Программа микроконтроллера 8
4.1. Блок-схема алгоритма 9
5. База данных 10
5.1. Просмотр содержимого таблиц 11
6. Обработка данных в Node-RED 12
7. Тестирование и проверка неисправности 13
8. Предложения по модернизации 14
9. Управление версиями Git 15
Выводы 16
Список используемых сокращений 17
Список используемой литературы 18
Приложение А. Структурная схема 19
Приложение Б. Принципиальная схема (EasyEDA) 20
Приложение В. Перечень элементов 21
Приложение Г. Схема в Wokwi 22
Приложение Д. Листинг программы Arduino 23
Приложение Е. ER-диаграмма и schema.sql 24
Приложение Ж. Поток Node-RED 25
Листинг функции разбора журнала 26

Введение

Современные системы мониторинга используют датчики, микроконтроллеры и программы для хранения показаний. Их задача — периодически измерять параметры среды и своевременно сообщать об отклонениях от заданных условий.

В курсовом проекте рассматривается виртуальная система контроля гаража на базе Arduino Uno. Система получает данные о концентрации газа, движении и положении ворот. При превышении порога концентрации включается реле вытяжки и светодиодная индикация.

Цель работы — разработать и проверить модель системы сбора и учёта данных по варианту №16. Принципиальная схема создаётся в EasyEDA, работа контроллера проверяется в Wokwi, показания сохраняются в SQLite, а загрузку и диагностику выполняет Node-RED.

Для достижения цели необходимо спроектировать схемы, написать программу Arduino, рассчитать основные параметры, создать базу данных, настроить обработку журнала и проверить штатные и нештатные сценарии.

Вся разработка выполняется виртуально. Принятая в работе шкала MQ-2 от 0 до 300 ppm представляет учебное линейное преобразование показаний АЦП, предусмотренное методическими указаниями.

1. Постановка задачи и структурная схема

Разрабатываемая система предназначена для контроля газовой обстановки в гараже. Она включает три входных датчика, контроллер Arduino Uno, релейный модуль вытяжки, красный светодиод и цепочку обработки показаний на компьютере.

Таблица 1 — Функции основных компонентов

Компонент	**Функция**
MQ-2 (s1)	Условная оценка концентрации газа, 0–300 ppm
PIR (s2)	Фиксация наличия движения, 0/1
SW1 (s3)	Контроль положения ворот, 0/1
Arduino Uno	Опрос каждые 5 с и управление
Реле и LED	Включение при s1 > 50 ppm
Node-RED и SQLite	Обработка журнала, хранение и контроль данных

При концентрации выше 50 ppm программа включает реле и светодиод, а при концентрации 50 ppm и ниже выключает их. События THRESHOLD и NORMAL отправляются в последовательный порт только при изменении состояния тревоги.

Для демонстрации работы предусмотрен дополнительный переключатель SW2: в режиме AUTO используется сигнал MQ-2, в режиме TEST подставляется заданное число из команды SET. При этом проверка порога остаётся общей для обоих режимов.

Вывод Serial Monitor переносится в data/log.txt. После сохранения файла поток Node-RED читает журнал, создаёт SQL-команды, проверяет диапазоны датчиков и обновляет базу SQLite. Структурная схема представлена в приложении А.

2. Принципиальная схема в EasyEDA и схема в Wokwi

Принципиальная схема выполнена в EasyEDA Standard. В её состав включены Arduino Uno R3, датчик MQ-2, PIR HC-SR501, два переключателя, одноканальное реле, красный светодиод и токоограничивающий резистор R1 сопротивлением 300 Ом.

Таблица 2 — Подключение к Arduino Uno

Компонент	**Подключение**	**Режим**
MQ-2, AOUT	A0	Аналоговый вход
PIR, OUT	D2	Цифровой вход
SW1, ворота	D3 → GND	INPUT_PULLUP
SW2, тестовый режим	D4 → GND	INPUT_PULLUP
Реле, IN	D7	Цифровой выход
LED через R1 = 300 Ом	D8 → R1 → анод LED; катод → GND	Цифровой выход
Модули	5V и GND	Питание

У MQ-2 используется аналоговый выход AOUT; цифровой DOUT не подключается. Для переключателей SW1 и SW2 включены внутренние подтягивающие резисторы: замыкание на GND считывается как LOW, после инвертирования соответствует логической единице.

Контакты COM, NO, NC релейного модуля оставлены свободными: в виртуальной модели проверяется срабатывание управляющего входа IN, без подключения реального вентилятора. Действительная система потребовала бы отдельной силовой цепи питания вытяжки.

Модель Wokwi собиралась по схеме EasyEDA с теми же сигналами A0, D2, D3, D4, D7 и D8. Проект модели: https://wokwi.com/projects/478582994646665217. Полный файл соединений хранится в sim/diagram.json. Схема EasyEDA и изображение модели приведены в приложениях Б и Г.

Перечень элементов (BOM) выгружается из EasyEDA отдельно в файл docs/bom.csv. В варианте с дополнительным SW2 в перечне восемь компонентов.

3. Расчётная часть

Расчёты выполнены по формулам из методических указаний.

3.1. Расчёт резистора светодиода

Исходные данные: напряжение питания U = 5 В; прямое падение напряжения на красном светодиоде Uf = 2,0 В; ток I = 10 мА = 0,01 А.

R = (U − Uf) / I = (5 − 2,0) / 0,01 = 300 Ом.

Вывод: выбирается R1 = 300 Ом. Резистор обеспечивает расчётный ток около 10 мА и применяется в схемах EasyEDA и Wokwi.

3.2. Расчёт шага АЦП

Arduino Uno имеет 10-битный АЦП (1024 уровня, коды 0–1023) с опорным напряжением 5 В.

ΔU = 5 / 1024 ≈ 0,00488 В = 4,88 мВ.

Для условного диапазона датчика s1 от 0 до 300 ppm шаг по шкале концентрации вычисляется согласно методичке:

ΔS = (300 − 0) / 1023 ≈ 0,293 ppm.

Вывод: 0,293 ppm соответствует одному изменению цифрового кода в учебной модели, но не определяет точность реального датчика MQ-2.

3.3. Расчёт объёма данных

Период опроса T = 5 с. Один цикл даёт три записи readings, условный объём одной записи равен 40 байт.

Число циклов за сутки: Nц = 86 400 / 5 = 17 280.

Число записей за сутки: N = 17 280 × 3 = 51 840.

Суточный объём: V = 51 840 × 40 = 2 073 600 байт ≈ 2,07 МБ.

Объём за 30 суток: V30 = 2 073 600 × 30 = 62 208 000 байт ≈ 62,21 МБ.

Вывод: без учёта событий и служебных данных SQLite расчётный объём показаний составит около 62,21 МБ за 30 дней непрерывной работы.

4. Программа микроконтроллера

Прошивка Arduino для варианта №16 разработана на языке C++. В setup() инициализируются порт Serial со скоростью 9600 бод, входы датчиков и выходы исполнительных устройств. Состояние реле и светодиода при запуске — выключено.

В каждом цикле loop() считываются MQ-2, PIR и SW1, после чего увеличивается номер измерения n. Аналоговый код MQ-2 преобразуется по формуле s1 = 300 × analogRead(A0) / 1023; s2 и s3 имеют значения 0/1. Между циклами выполняется задержка 5000 мс.

Тестовое значение задаётся командой SET, полученной через Serial Monitor, но сам режим TEST включается только отдельным переключателем SW2 (D4). В обоих режимах применяется одна и та же проверка порога, а выбранный режим включается в каждую строку журнала полем mode.

Таблица 3 — Формат сообщений Serial Monitor

Тип	**Пример**	**Назначение**
DATA	DATA n=7 s1=30.00 s2=0 s3=1 mode=TEST	Результат одного цикла
EVENT	EVENT n=7 type=NORMAL	Изменение состояния тревоги
EVENT	EVENT n=11 type=THRESHOLD mode=TEST	Превышение порога

Чтобы не создавать одинаковые события в каждом цикле, используется переменная alarm. При переходе из нормального состояния в тревожное выводится THRESHOLD, при обратном переходе — NORMAL. Программная логика не содержит гистерезиса: включение происходит при s1 > 50 ppm, выключение — при s1 ≤ 50 ppm.

Листинг прошивки помещён в приложение Д. Основной файл программы хранится в firmware/variant_16.ino.

4.1. Блок-схема алгоритма

Алгоритм работы в режимах AUTO и TEST показан на рисунке 1.

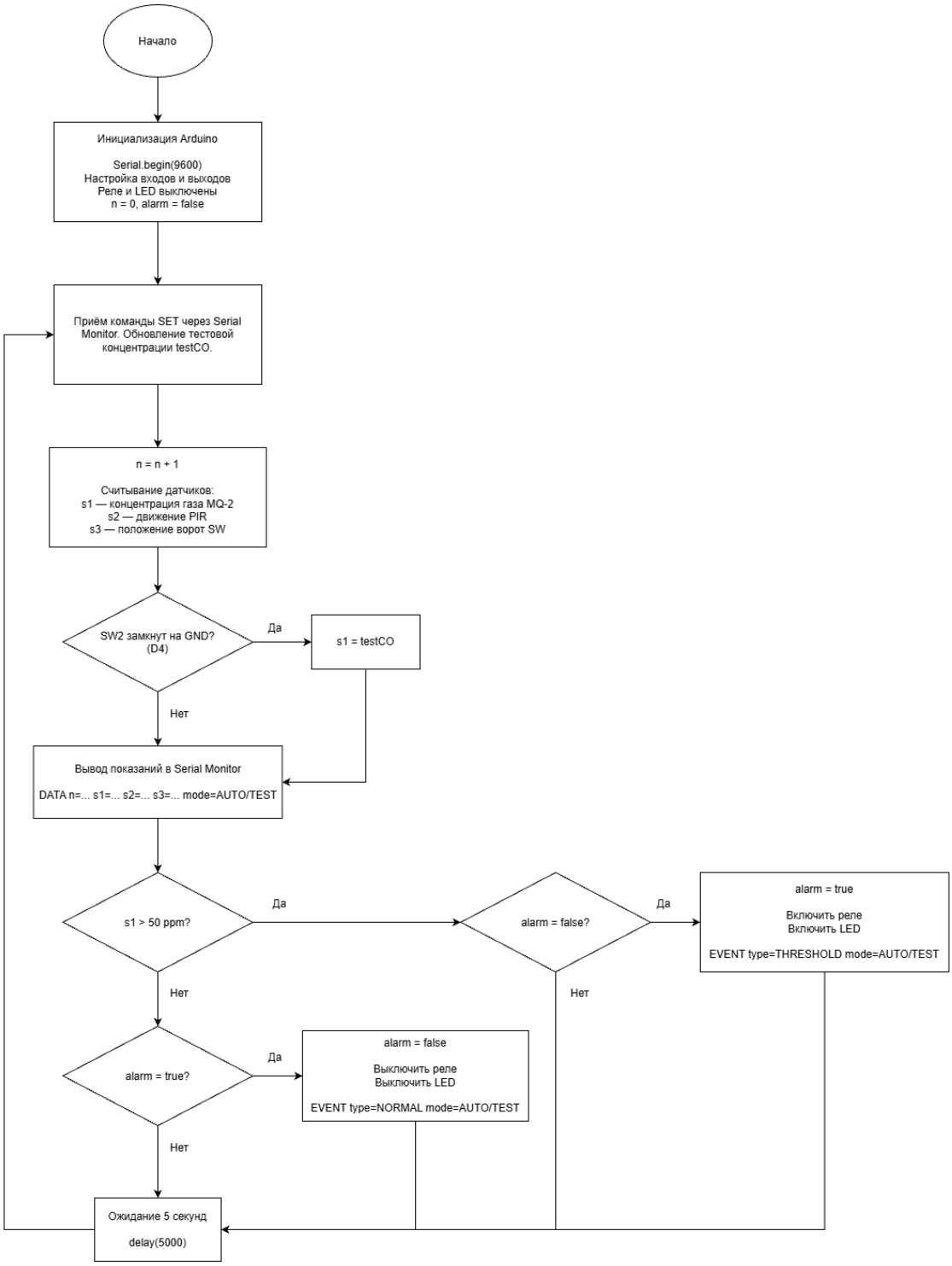


Рисунок 1 — Блок-схема программы Arduino

5. База данных

Для хранения показаний создана база SQLite db/variant.db. В ней используются две таблицы: readings — отдельные показания датчиков; events — события управления и неисправностей. Структура задана файлом db/schema.sql.

Таблица 4 — Таблицы базы данных

Таблица	**Поля**	**Назначение**
readings	id, n, t_s, sensor, value, mode	Три записи на каждый цикл измерения
events	id, n, t_s, type, message, mode	События THRESHOLD, NORMAL, FAULT

Поле id служит первичным ключом. Поле n содержит номер измерения, а t_s — время в секундах, рассчитанное по формуле $t_s = n \times 5$. Значение mode фиксирует AUTO либо TEST. Поле message используется для пояснения причины FAULT.

Формальная связь внешним ключом между readings и events не задана. Логическое сопоставление записей выполняется по номеру n и времени t_s. ER-диаграмма и SQL-скрипт создания таблиц приведены в приложении Е.

В файле db/queries.sql подготовлены четыре проверочных запроса: минимальное, среднее и максимальное значения по каждому датчику (отдельно по режимам); список событий по времени; количество записей в таблицах; десять последних показаний. Запросы из методички и раздаточной заготовки таким образом учтены.

```
sqlite3 db\variant.db ".read db\schema.sql"
sqlite3 db\variant.db ".read db\queries.sql"
```

5.1. Просмотр содержимого таблиц

Для проверки сохранённых данных выполнены запросы SELECT к таблицам readings и events. Результаты запросов показаны на рисунках 2 и 3.

```
SQLite-3.54 variant.db-> SELECT n, t_s, sensor, value, mode
                    ;-> FROM readings ORDER BY n, sensor;
n  t_s  sensor  value   mode
-  ---  ------  ------  ----
1    5  s1        30.0  TEST
1    5  s2         0.0  TEST
1    5  s3         1.0  TEST
2   10  s1        30.0  TEST
2   10  s2         0.0  TEST
2   10  s3         1.0  TEST
3   15  s1      265.98  AUTO
3   15  s2         0.0  AUTO
3   15  s3         1.0  AUTO
```

Рисунок 2 — Содержимое таблицы readings

В таблице readings содержатся девять записей: по три показания для измерений n = 1, 2 и 3. Для n = 1 и n = 2 сохранён режим TEST и значение s1 = 30 ppm; для n = 3 — режим AUTO и значение s1 = 265,98 ppm. Значения времени t_s равны 5, 10 и 15 с и соответствуют формуле t_s = n × 5.

```
SQLite-3.54 variant.db-> SELECT n, t_s, type, message, mode
                    ·-> FROM events ORDER BY t_s, id;
n  t_s    type      message  mode
-  ---  ---------  -------  ----
3   15  THRESHOLD           AUTO
```

Рисунок 3 — Содержимое таблицы events

В таблице events сохранено событие THRESHOLD для измерения n = 3, времени 15 с и режима AUTO. Оно соответствует превышению порога 50 ppm при s1 = 265,98 ppm. Поле message для этого события не заполнено.

6. Обработка данных в Node-RED

Для загрузки журнала создан поток Node-RED. Он запускается из папки проекта и использует узлы inject, watch, read file, function, write file и exec. Конфигурация экспортирована в nodered/flow.json.

Таблица 5 — Основные узлы потока Node-RED

Узел	**Действие**
watch	Обнаруживает сохранение data/log.txt
read file	Считывает журнал одной строкой UTF-8
function	Разбирает DATA/EVENT, проверяет диапазоны, формирует SQL
write file	Перезаписывает db/load.sql
exec	Запускает sqlite3 для загрузки SQL-команд
debug	Выводит данные и ошибки для проверки
inject	Ручной запуск чтения data/log.txt

Функция разбора использует период T = 5 с и диапазоны s1: 0–300 ppm, s2: 0–1, s3: 0–1. При трёх числовых показаниях строка DATA даёт три записи readings. Строки EVENT создают записи с типом THRESHOLD или NORMAL. При отсутствующем либо недопустимом значении датчика создаётся дополнительное событие FAULT.

Поле mode берётся из строки журнала и сохраняется в базе. Текст вида s1=30.00 преобразуется в значение датчика; при обработке записей TEST не изменяется алгоритм контроля диапазонов.

Для защиты от дублирования при повторной загрузке формируется SQL-транзакция с очисткой текущих данных и повторным заполнением таблиц по всему журналу. Таким образом, файл data/log.txt выступает источником полного набора записей, а не журналом накопления с дозаписью в базу.

Файл load.sql создаётся автоматически. Для проверки достаточно сохранить data/log.txt, затем выполнить контрольные запросы в SQLite. Схема потока и место хранения его экспорта приведены в приложении Ж.

7. Тестирование и проверка неисправности

Проверка проводилась в Wokwi и SQLite. Для воспроизведения концентраций ниже 50 ppm применяется тестовый переключатель SW2 и команда SET. В штатном режиме система использует показание MQ-2. Режимы фиксируются в журнале полем mode.

Таблица 6 — Результаты проверок

Проверка	**Ожидаемый результат**	**Полученный результат**
AUTO, s1 = 265,98 ppm	Включение реле и LED; THRESHOLD	Выполнено
TEST, SET 30	Выключение реле и LED; NORMAL	Выполнено
TEST, SET 80	Включение; THRESHOLD	Выполнено
Повторное сохранение журнала	Без дублирования записей	9 записей осталось 9
Неверное значение s1 = 350	Событие FAULT	Создано
350 ppm и отдельное EVENT THRESHOLD	Две записи events	FAULT и THRESHOLD

Проверка диагностирования проводилась на искусственно подготовленной строке «DATA n=20 s1=350.00 s2=0 s3=1 mode=TEST». Так как 350 ppm превышает допустимые 300 ppm, Node-RED создал событие FAULT. Когда в журнал дополнительно внесена строка EVENT n=20 type=THRESHOLD mode=TEST, база сохранила оба события независимо.

В тестовом журнале с одной строкой DATA были получены три записи readings и одно событие events. При добавлении строки EVENT стало два события. Эти данные проверены запросом SELECT к таблице events.

```
C:\Users\Alexey\Moiseev>sqlite3 db\variant.db "SELECT n, type, message, mode FROM events;"
```

n	type	message	mode
20	FAULT	датчик s1 вне диапазона	TEST
20	THRESHOLD	NULL	TEST

```
C:\Users\Alexey\Moiseev>
```

Рисунок 4 — Результат проверки FAULT и THRESHOLD

8. Предложения по модернизации

По результатам разработки предлагаются три направления развития системы. Они относятся к возможной модернизации и не изменяют выполненную основную реализацию без согласования с преподавателем.

8.1. Гистерезис управления вытяжкой

Чтобы исключить частые включения и выключения реле при колебаниях концентрации около 50 ppm, предлагается включать вытяжку при CO > 50 ppm, выключать при CO ≤ 40 ppm, а при 40 < CO ≤ 50 ppm оставлять предыдущее состояние. Это позволит уменьшить количество переключений реле и его износ.

В текущей прошивке оставлено исходное правило методички: включение при CO > 50 ppm и выключение при CO ≤ 50 ppm. Внедрение гистерезиса возможно после согласования с руководителем.

8.2. Применение специализированного датчика CO

В учебной модели применён MQ-2 по заданию варианта. Для более достоверного контроля именно угарного газа в реальном гараже можно использовать специализированный датчик CO с необходимой калибровкой и проверкой измерительного канала.

8.3. Автоматическое получение данных

Сейчас строки Serial Monitor копируются из браузерной модели в data/log.txt вручную. При работе с физической Arduino можно передавать данные с USB-порта непосредственно в Node-RED через последовательный интерфейс и затем сохранять их в SQLite. Это сократит ручные действия и вероятность ошибок копирования.

9. Управление версиями Git

Файлы проекта ведутся в репозитории Git. В папках firmware, sim, db, nodered, data и docs хранятся программа Arduino, модель, скрипты базы данных, поток обработки, журналы и документация. Изменения сохраняются коммитами по этапам разработки.

Репозиторий проекта: https://github.com/Crypton359/Moiseev_Coursework

В ветке main репозитория сохранены шесть коммитов по этапам разработки:

86628ef — Initial project structure
41ddb50 — Add system structure diagram
09c4438 — Add electrical schematic and BOM
0161334 — Add Arduino firmware and complete circuit diagrams
2b23640 — Add engineering calculations
Last commit — Finalize coursework project

Для проверки состояния файлов используется git status, для просмотра истории — git log --oneline. После внесения изменений выполняются git add -A и git commit с описанием этапа. Для отправки сохранённых изменений на GitHub используется git push -u origin main.

Выводы

Приведённые результаты подтверждают работу управления по порогу и обработки неисправностей средствами Node-RED. В реальном применении датчик CO потребовал бы отдельного подбора и калибровки.

В курсовом проекте по варианту №16 разработана виртуальная система контроля гаража на базе Arduino Uno. Подготовлены структурная и принципиальная схемы, модель Wokwi, программа контроллера, база данных SQLite и поток обработки Node-RED.

Программа опрашивает MQ-2, PIR и переключатель ворот с периодом 5 секунд. При превышении учебного порога 50 ppm включаются реле и красный светодиод; при возвращении в норму они выключаются. Для демонстрации предусмотрен режим TEST с выбором отдельным переключателем SW2.

Показания и события передаются в журнал, после сохранения автоматически обрабатываются Node-RED и загружаются в SQLite. Подтверждены регистрация THRESHOLD/NORMAL, проверка FAULT и повторная загрузка без дублирования. Три расчёта и проверочные SQL-запросы выполнены.

В качестве дальнейших улучшений предложены гистерезис управления вытяжкой, специализированный датчик CO и автоматизированное получение данных. Реализованная система соответствует учебной задаче виртуального сбора, хранения и контроля показаний.

Список используемых сокращений

АЦП — аналого-цифровой преобразователь.

CO — угарный газ.

PIR — пассивный инфракрасный датчик движения.

LED — светодиод.

SQL — язык структурированных запросов.

SQLite — встраиваемая система управления базами данных.

UART — последовательный интерфейс передачи данных.

Git — система контроля версий.

ppm — частей на миллион (единица концентрации).

Список используемой литературы

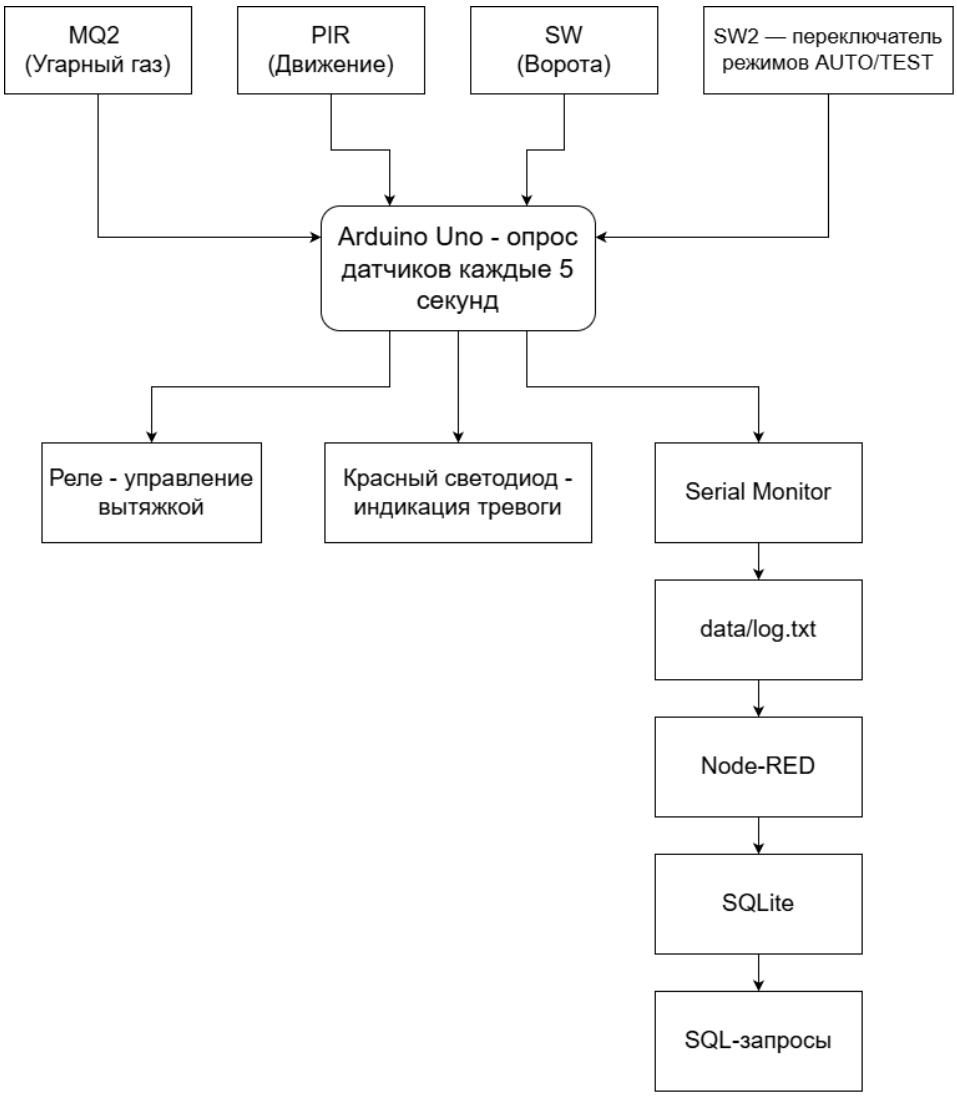
1. Документация Wokwi. — URL: https://docs.wokwi.com (дата обращения: 24.10.2026).

2. Справочник языка Arduino. — URL: https://www.arduino.cc/reference/ (дата обращения: 24.10.2026).

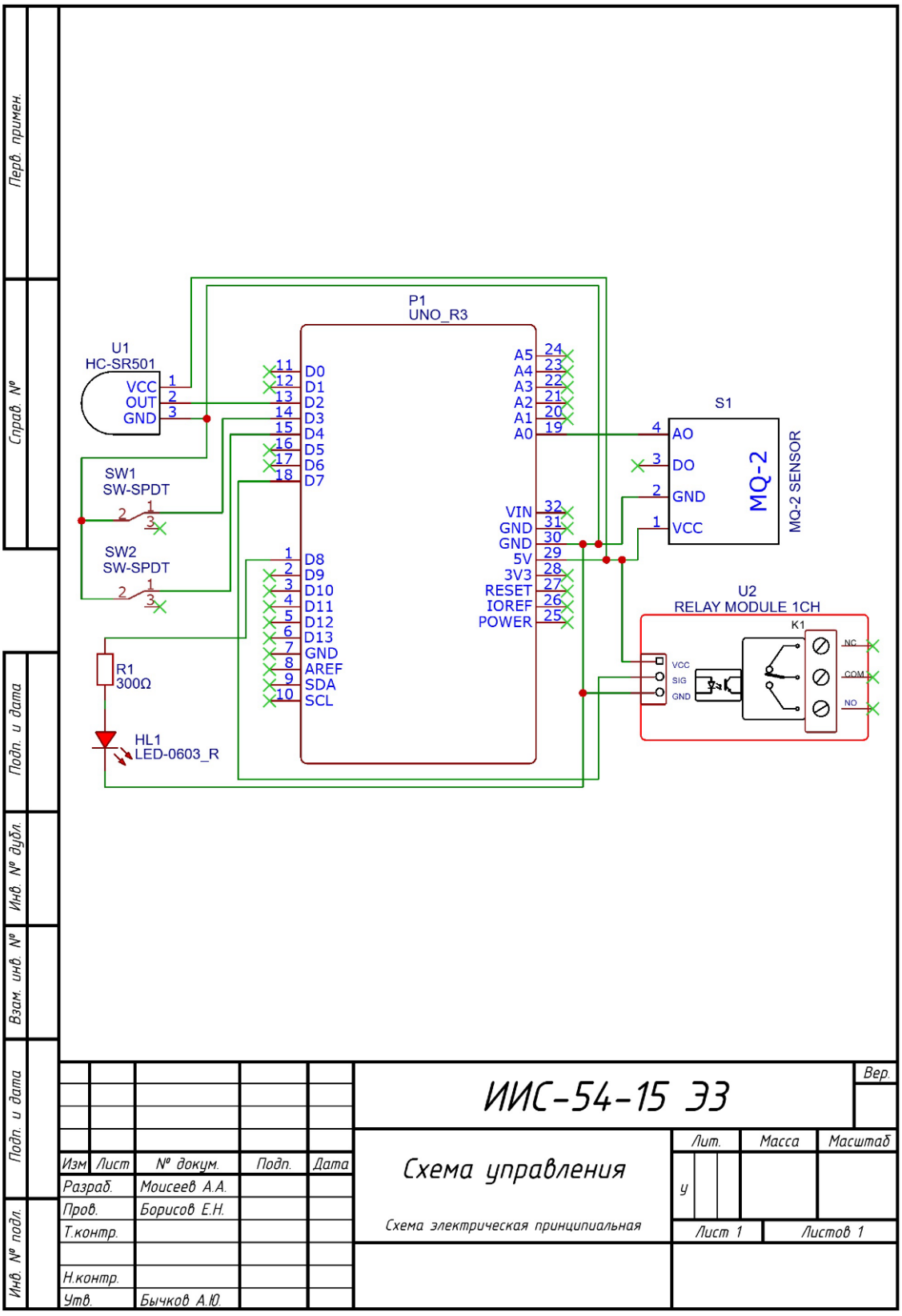
3. Документация Node-RED. — URL: https://nodered.org/docs/ (дата обращения: 24.10.2026).

4. Документация SQLite. — URL: https://www.sqlite.org/docs.html (дата обращения: 24.10.2026).

Приложение А. Структурная схема



Приложение Б. Принципиальная схема (EasyEDA)



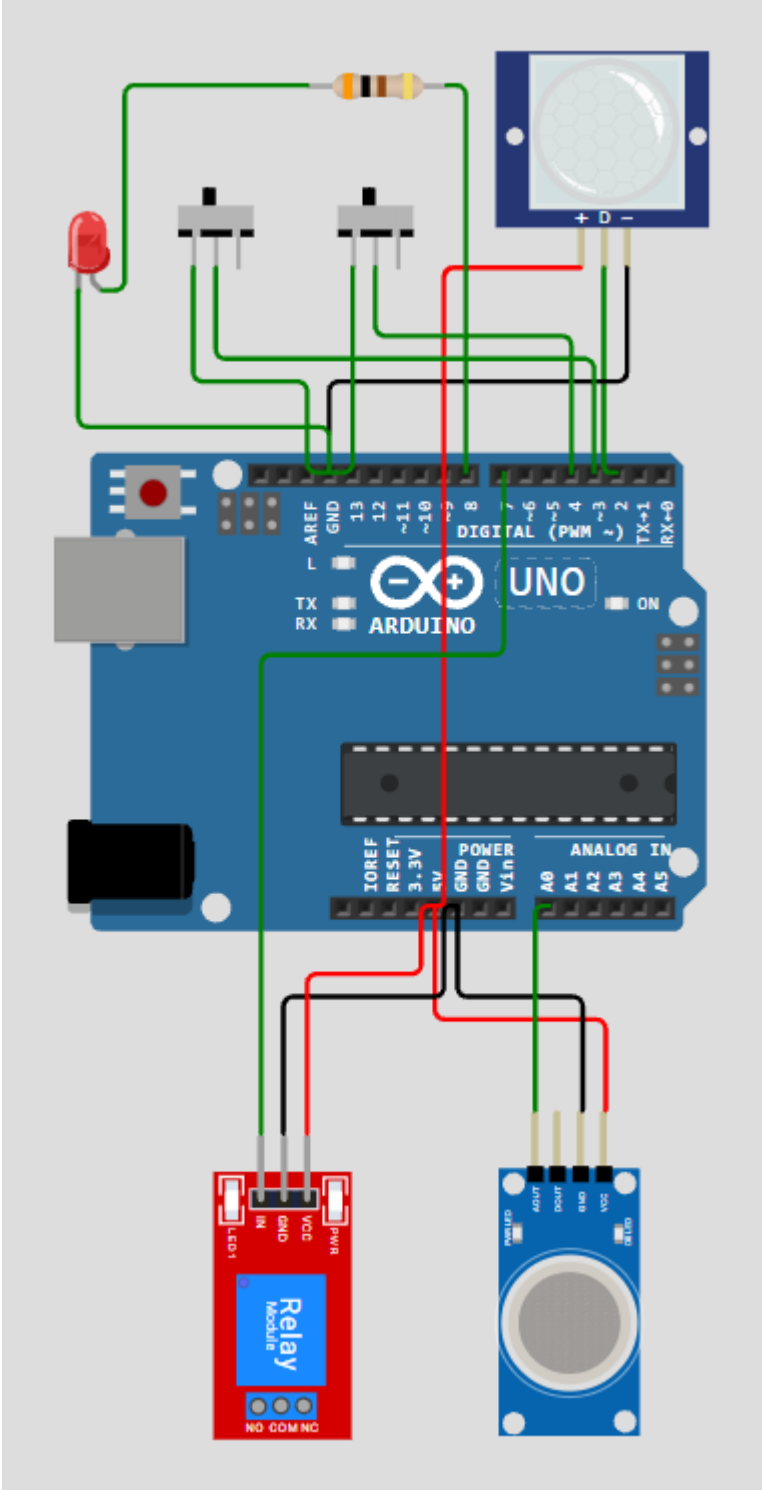
Приложение В. Перечень элементов

Поз. обозначение	Наименование	Кол.	Примечание
	Приборы световой сигнализации		
HL1	LED-0603_R	1	
	Приборы, измерительное оборудование		
P1	UNO_R3	1	
	Резисторы		
R1	300Ω	1	
	Коммутационные устройства		
S1	Sensor_MQ-2_module_vertical_AlH	1	
	Переключатели		
SW1, SW2	SW-SPDT	2	
	Преобразователи электрических величин		
U1	HC-SR501	1	
U2	RELAY MODULE 1CH	1	

Перв. примен. | Справ. № | Подп. и дата | Инв. № дубл. | Взам. инв. № | Подп. и дата | Инв. № подл.

Изм	Лист	№ докум.	Подп.	Дата	ИИС-54-15 ПЭЗ	Вер.
Разраб.		Моисеев А.А.			Схема управления	Лит. У	Лист 1	Листов 1
Пров.		Борисов Е.Н.				
Нач. сект.						
Н.контр.						
Утв.		Бычков А.Ю.			Перечень элементов	

Приложение Г. Схема в Wokwi



Приложение Д. Листинг программы Arduino

```
const int PIN_MQ2 = A0;
const int PIN_PIR = 2;
const int PIN_SW = 3;
const int PIN_TEST_SW = 4;
const int PIN_RELAY = 7;
const int PIN_LED = 8;
const int T = 5000;
const float THRESHOLD = 50.0;
unsigned long n = 0;
bool alarm = false;
float testCO = 30.0;
void setup() {
  Serial.begin(9600);
  pinMode(PIN_PIR, INPUT);
  pinMode(PIN_SW, INPUT_PULLUP);
  pinMode(PIN_TEST_SW, INPUT_PULLUP);
  pinMode(PIN_RELAY, OUTPUT);
  pinMode(PIN_LED, OUTPUT);
  digitalWrite(PIN_RELAY, LOW);
  digitalWrite(PIN_LED, LOW);
}
void loop() {
  if (Serial.available() > 0) {
    String command = Serial.readStringUntil('\n');
    command.trim();
    if (command.startsWith("SET ")) {
      float value = command.substring(4).toFloat();
      if (value >= 0 && value <= 300) {
        testCO = value;
      }
    }
  }
  n++;
  bool testMode = !digitalRead(PIN_TEST_SW);
  int raw = analogRead(PIN_MQ2);
  float s1 = 300.0 * raw / 1023.0;
  if (testMode) s1 = testCO;
  int s2 = digitalRead(PIN_PIR);
  int s3 = !digitalRead(PIN_SW);
  Serial.print("DATA n="); Serial.print(n);
  Serial.print(" s1="); Serial.print(s1, 2);
  Serial.print(" s2="); Serial.print(s2);
  Serial.print(" s3="); Serial.print(s3);
  Serial.print(" mode=");
  Serial.println(testMode ? "TEST" : "AUTO");
  bool danger = s1 > THRESHOLD;
  if (danger && !alarm) {
    alarm = true;
    digitalWrite(PIN_RELAY, HIGH);
    digitalWrite(PIN_LED, HIGH);
    Serial.print("EVENT n="); Serial.print(n);
    Serial.print(" type=THRESHOLD");
    Serial.println(testMode ? " mode=TEST" : " mode=AUTO");
  } else if (!danger && alarm) {
    alarm = false;
    digitalWrite(PIN_RELAY, LOW);
    digitalWrite(PIN_LED, LOW);
    Serial.print("EVENT n="); Serial.print(n);
    Serial.print(" type=NORMAL");
    Serial.println(testMode ? " mode=TEST" : " mode=AUTO");
  }
  delay(T);
}
```

Приложение E. ER-диаграмма и schema.sql

readings	
id	INTEGER PK
n	INTEGER
t_s	INTEGER
sensor	TEXT
value	REAL
mode	TEXT

events	
id	INTEGER PK
n	INTEGER
t_s	INTEGER
type	TEXT
message	TEXT
mode	TEXT

```
CREATE TABLE IF NOT EXISTS readings (
  id INTEGER PRIMARY KEY AUTOINCREMENT,
  n INTEGER NOT NULL,
  t_s INTEGER NOT NULL,
  sensor TEXT NOT NULL,
  value REAL NOT NULL,
  mode TEXT NOT NULL DEFAULT 'AUTO'
);
CREATE TABLE IF NOT EXISTS events (
  id INTEGER PRIMARY KEY AUTOINCREMENT,
  n INTEGER NOT NULL,
  t_s INTEGER NOT NULL,
  type TEXT NOT NULL,
  message TEXT,
  mode TEXT NOT NULL DEFAULT 'AUTO'
);
```

Приложение Ж. Поток Node-RED

Поток сохранён в файле nodered/flow.json. Его назначение — чтение data/log.txt, подготовка db/load.sql и запуск sqlite3 для загрузки данных.

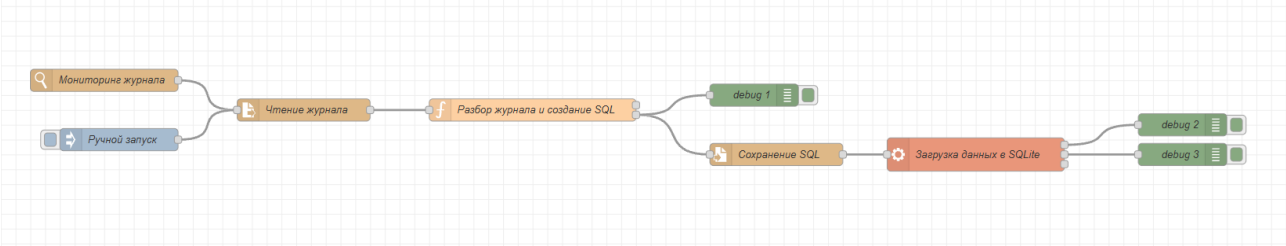


Схема соединения узлов: watch → read file → function → write file → exec. Узел inject подключён к read file для ручного запуска. Диагностика выводится в debug. Функция добавляет записи в readings и events командами INSERT и формирует FAULT при недопустимом показании.

Экспорт потока и исходные скрипты хранятся в репозитории проекта. Загрузка запускается при сохранении data/log.txt либо кнопкой узла «Ручной запуск».

Листинг функции разбора журнала

Исходный код узла function из файла nodered/flow.json проекта.

```
// Период опроса в секундах
const T = 5;

// Допустимые диапазоны датчиков
const RANGES = {
    s1: [0,300],
    s2: [0, 1],
    s3: [0, 1]
};

// Получаем текст журнала
const text = String(msg.payload).replace(/^\uFEFF/, '');
const sql = [];

// Обрабатываем строки
text.split(/\r?\n/).forEach(raw => {
    const line = raw.trim();
    if (!line) return;

    const parts = line.split(/\s+/);
    const kind = parts[0];

    // Получаем параметры вида имя=значение
    const f = {};

    parts.slice(1).forEach(p => {
        const index = p.indexOf('=');

        if (index > 0) {
            f[p.slice(0, index)] = p.slice(index + 1);
        }
    })

    // Проверяем номер измерения
    if (!/^\d+$/.test(f.n || '')) return;

    const n = Number(f.n);
    if (n < 1 || !Number.isSafeInteger(n)) return;

    const t = n * T;

    // Определяем режим измерения
    const mode = f.mode === 'TEST' ? 'TEST' : 'AUTO';

    // Обработка показаний датчиков
    if (kind === 'DATA') {
        ['s1', 's2', 's3'].forEach(s => {
            const rawValue = f[s];
            const validNumber = rawValue !== undefined && /^-?\d+(\.\d+)?
$/.test(rawValue);

            const v = validNumber ? Number(rawValue) : NaN;

            // Сохраняем корректное числовое показание
            if (Number.isFinite(v)) {
                sql.push(
                    `INSERT INTO readings(n,t_s,sensor,value,mode) VALUES(${n},$
{t},'${s}',${v},'${mode}');`
                );
            }

            // Проверяем неисправность датчика
            const [min, max] = RANGES[s];

            if (!Number.isFinite(v) || v < min || v > max) {
```

```
                        sql.push(
                            `INSERT INTO events(n,t_s,type,message,mode) VALUES(${n},${t},'FAULT','датчик ${s} вне диапазона','${mode}');`
                        );
                    }
                });
            }

            // Обработка событий Arduino
            else if (kind === 'EVENT') {
                if (f.type === 'THRESHOLD' || f.type === 'NORMAL') {
                    sql.push(
                        `INSERT INTO events(n,t_s,type,message,mode) VALUES(${n},${t},'${f.type}',NULL,'${mode}');`
                    );
                }
            }

        });

        // Если подходящих записей нет, ничего не загружаем
        if (sql.length === 0) {
            node.warn("В журнале нед данных для загрузки");
            return [null, null];
        }

        // Формируем SQL-файл
        const result = 'BEGIN;\n' + 'DELETE FROM readings;\n' + 'DELETE FROM events;\n' + "DELETE FROM sqlite_sequence WHERE name IN ('readings','events');\n" + sql.join('\n') + '\nCOMMIT;\n';

        // Передаем SQL через второй выход
        return [null, {payload: result}];
```